



Plataforma de Catálogo, Compra e Aluguel de Filmes

Integrantes

- Marcos Filipe de Paula Macêdo
 - Tailon Cypreste
 - Pedro Henrique Amorim
-



Descrição Geral do Produto

O CrazyCine é uma plataforma web desenvolvida utilizando Flask com o objetivo de permitir que usuários possam explorar um catálogo de filmes, criar listas personalizadas, marcar filmes como assistidos e simular compras e aluguéis utilizando pagamento via Pix.

O sistema foi desenvolvido com foco em organização, experiência do usuário e aplicação prática dos conceitos de desenvolvimento web, banco de dados, autenticação e integração com APIs externas.

A plataforma possui interface responsiva e moderna, permitindo acesso tanto em computadores quanto em dispositivos móveis.



Funcionalidades do Sistema

Autenticação de Usuários

- Cadastro de usuários
- Login
- Logout

- Controle de sessão
-

Catálogo de Filmes

- Visualização de filmes cadastrados
 - Busca por nome
 - Filtro por gênero
 - Página de detalhes dos filmes
-

Sistema de Listas

- Criação de listas personalizadas
 - Adição de filmes às listas
 - Remoção de filmes das listas
 - Organização pessoal de catálogo
-

Controle de Filmes Assistidos

- Marcar filmes como assistidos
 - Remover marcação
 - Histórico pessoal do usuário
-

Sistema de Compra e Aluguel

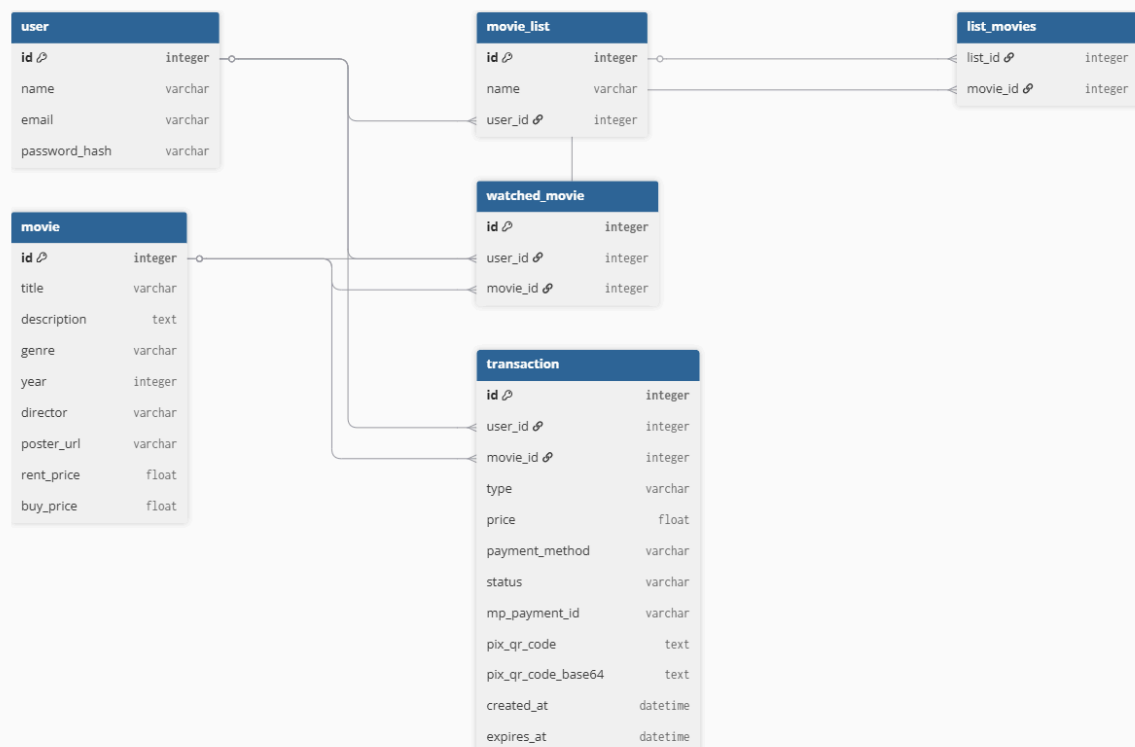
- Simulação de compra
 - Simulação de aluguel
 - Integração com Mercado Pago
 - Controle de validade de aluguel
 - Histórico de transações
-

Dashboard do Usuário

- Quantidade de listas criadas
- Quantidade de filmes assistidos
- Quantidade total de filmes
- Quantidade de pedidos realizados

Diagrama do Banco de Dados

Segue abaixo o modelo ER (Entidade Relacionamento) utilizado no desenvolvimento do sistema:



 dbdiagram.io

Explicação da Arquitetura Adotada

O projeto foi desenvolvido utilizando uma arquitetura baseada no padrão MVC adaptado ao Flask.

Models

Responsáveis pela representação das entidades do banco de dados.

Exemplos:

- User
- Movie
- MovieList
- Transaction
- WatchedMovie

Os models foram desenvolvidos utilizando SQLAlchemy ORM.

Views (Templates)

Responsáveis pela interface visual do sistema.

Os templates foram desenvolvidos utilizando:

- HTML5
- CSS3
- Bootstrap 5
- Jinja2

Principais páginas:

- Home
- Login
- Cadastro
- Dashboard
- Catálogo de filmes
- Detalhes do filme

- Listas do usuário
-

Controllers (Blueprints)

Os Blueprints foram utilizados para separar responsabilidades dentro da aplicação.

Blueprints utilizados

- Main
- Auth
- Movies
- Lists
- Payments

Essa separação melhora:

- Organização
 - Escalabilidade
 - Reutilização de código
 - Manutenção
-



Detalhamento Técnico do Código

Sistema de Busca de Filmes

Trecho responsável pela busca dinâmica de filmes por nome.

```
query = request.args.get('q', '').strip()

if query:
    movies_query = movies_query.filter(
```

```
Movie.title.ilike(f'%{query}%')
)
```

A busca utiliza `ilike`, permitindo pesquisas sem diferenciação entre letras maiúsculas e minúsculas.

Sistema de Filtro por Gênero

```
genre = request.args.get('genre', '').strip()

if genre:
    movies_query = movies_query.filter(
        Movie.genre.ilike(f'%{genre}%')
    )
```

Permite filtrar filmes dinamicamente através da interface.

Controle de Filmes Assistidos

```
watched = WatchedMovie.query.filter_by(
    user_id=session['user_id'],
    movie_id=movie_id
).first()
```

Esse trecho verifica se determinado filme já foi marcado como assistido pelo usuário.

Controle de Aluguel Ativo

```
active_rental = Transaction.query.filter(
    Transaction.user_id == session['user_id'],
    Transaction.movie_id == movie_id,
    Transaction.type == 'aluguel',
    Transaction.status == 'aprovado',
```

```
Transaction.expires_at > datetime.utcnow()  
) .first()
```

Responsável por validar se o usuário possui um aluguel ainda ativo.

Seed Automática de Filmes

```
movie.poster_url = poster  
movie.description = desc  
movie.rent_price = rent  
movie.buy_price = buy
```

O sistema possui uma seed automática que popula o banco com filmes hospedadas pela API do TMDB (The Movie Database)..

Como Rodar o Projeto

1. Clonar o repositório

```
git clone https://github.com/Marcos-Macedo444/CrazyCine-Flask.git
```

2. Entrar na pasta

```
cd CrazyCine-Flask
```

3. Criar ambiente virtual

```
python -m venv venv
```

4. Ativar ambiente virtual

Windows

```
venv\Scripts\activate
```

Linux/Mac

```
source venv/bin/activate
```

5. Instalar dependências

```
pip install -r requirements.txt
```

6. Configurar variáveis de ambiente

Exemplo:

```
SECRET_KEY=sua_chave  
DATABASE_URL=seu_banco  
WEBHOOK_BASE_URL=url_do_render
```

7. Executar o projeto

```
python run.py
```

8. Acessar no navegador

```
http://127.0.0.1:5000
```


Principais Desafios Encontrados

Organização do Projeto

Inicialmente o projeto possuía muitas funcionalidades concentradas em poucos arquivos.

Solução

Foi adotada a arquitetura baseada em Blueprints para separar responsabilidades e melhorar a manutenção.

Integração com Mercado Pago

A integração exigiu configuração de credenciais, tratamento de respostas da API e gerenciamento de estados de pagamento.

Solução

Foi criada uma estrutura específica para gerenciamento de pagamentos e validação de transações.

Deploy no Render

Durante o deploy surgiram problemas relacionados a:

- Variáveis de ambiente
- Banco de dados
- Rotas
- URLs externas

Solução

As variáveis de ambiente foram configuradas corretamente no Render e a estrutura do projeto foi ajustada para ambiente de produção.

Controle de Sessão

Foi necessário garantir que apenas usuários autenticados acessassem determinadas funcionalidades.

Solução

Foi desenvolvido um decorator personalizado utilizando Flask Session para validação de login.



Conclusão

O CrazyCine permitiu aplicar na prática conceitos fundamentais de desenvolvimento web utilizando Flask, SQLAlchemy, Bootstrap e integração com APIs externas.

O projeto proporcionou experiência prática em:

- Arquitetura de software
- Banco de dados
- Autenticação
- Organização de código
- Deploy
- Integração com pagamentos
- Desenvolvimento Full Stack

Além disso, o sistema foi estruturado de forma escalável e organizada, facilitando futuras melhorias e expansões.